



中国科学技术大学
University of Science and Technology of China

计算机系统概论A

Introduction to Computing Systems
(CS1002A.03)

Chapter 16 Pointers and Arrays

陈俊仕

cjuns@ustc.edu.cn
2025 Fall

计算机科学与技术学院
School of Computer Science and Technology



1 Review

2 Pointers

3 Arrays

4 Summary



1 Review

2 Pointers

3 Arrays

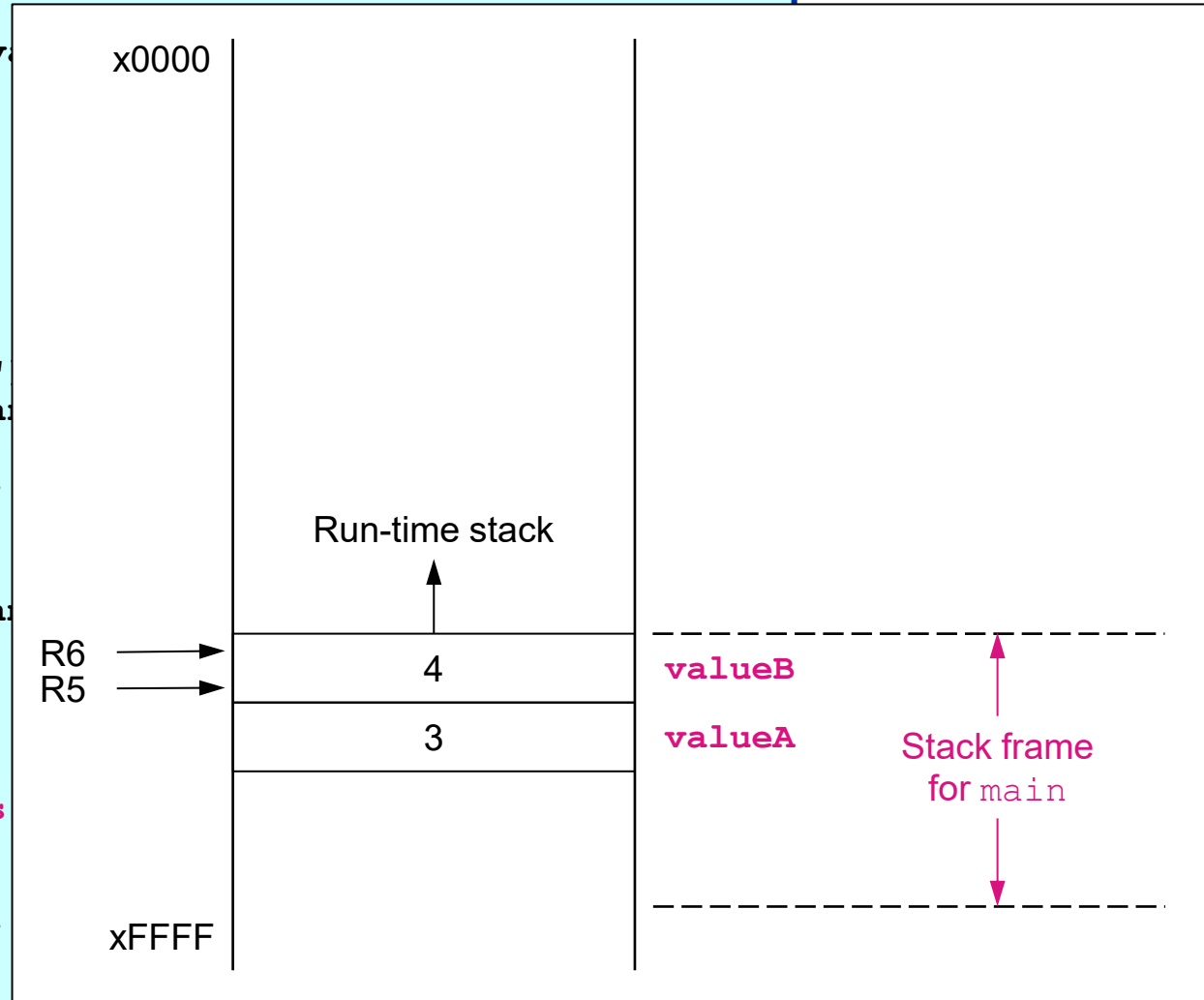
4 Summary

A classic example -- 'Swap'

```
1  #include <stdio.h>
2
3  // Swap has no return value thus is declared as void
4  void Swap(int firstVal, int secondVal);
5
6  int main(void)
7  {
8      int valueA = 3;
9      int valueB = 4;
10
11     printf("Before Swap ");
12     printf("valueA = %d and valueB = %d\n", valueA, valueB);
13
14     Swap(valueA, valueB);
15
16     printf("After Swap ");
17     printf("valueA = %d and valueB = %d\n", valueA, valueB);
18 }
19
20 void Swap(int firstVal, int secondVal)
21 {
22     int tempVal; // Holds firstVal when swapping
23
24     tempVal = firstVal;
25     firstVal = secondVal;
26     secondVal = tempVal;
27 }
```

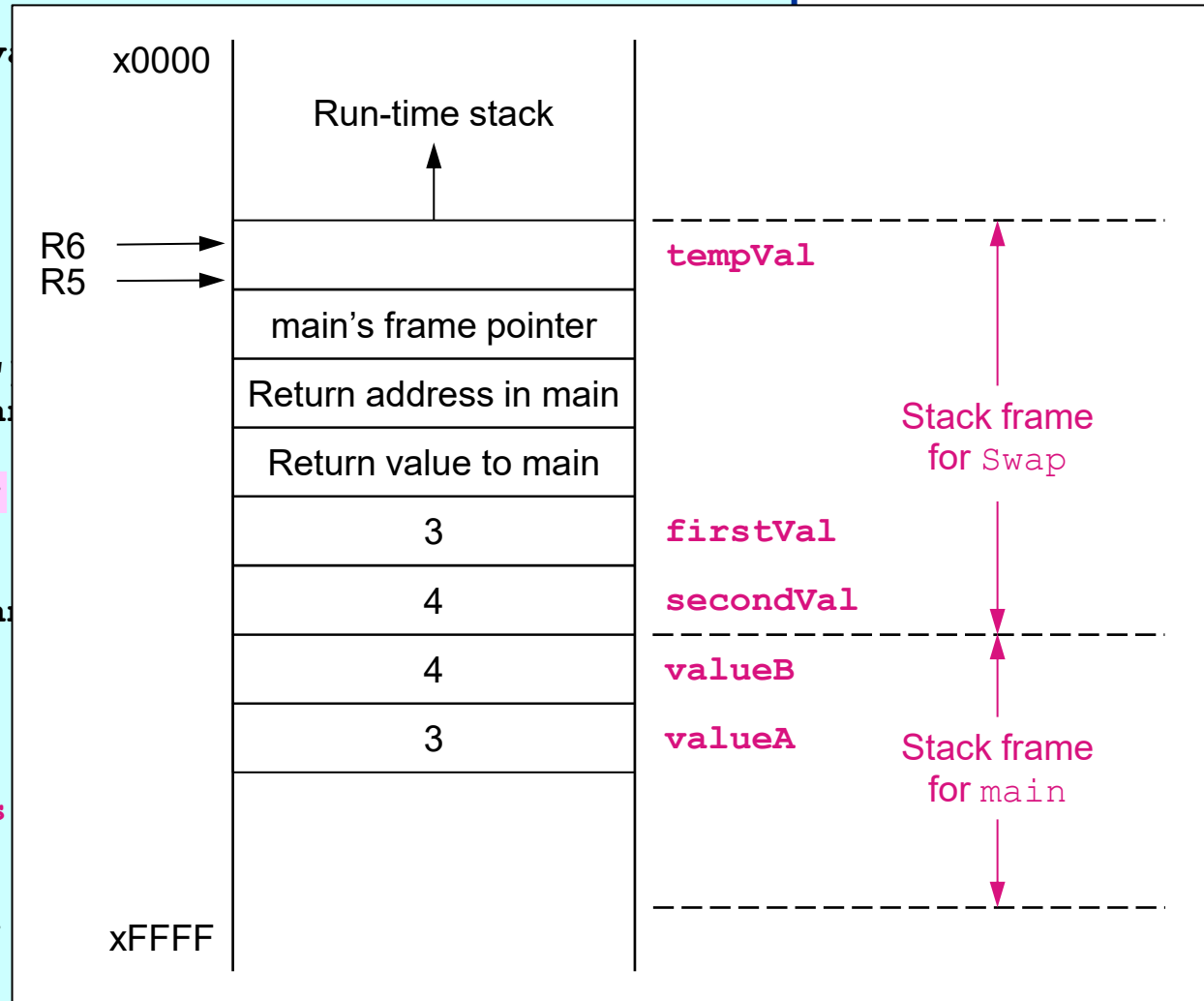
A classic example -- 'Swap'

```
1  #include <stdio.h>
2
3  // Swap has no return value
4  void Swap(int firstVal,
5           int secondVal)
6  {
7      int tempVal;
8      tempVal = firstVal;
9      firstVal = secondVal;
10     secondVal = tempVal;
11 }
12
13 int main(void)
14 {
15     int valueA = 3;
16     int valueB = 4;
17
18     printf("Before Swap\n");
19     printf("valueA = %d and valueB = %d\n", valueA, valueB);
20
21     Swap(valueA, valueB);
22
23     printf("After Swap\n");
24     printf("valueA = %d and valueB = %d\n", valueA, valueB);
25 }
26
27 void Swap(int firstVal,
28          int secondVal)
29 {
30     int tempVal; // Holds the value of firstVal
31     tempVal = firstVal;
32     firstVal = secondVal;
33     secondVal = tempVal;
34 }
```



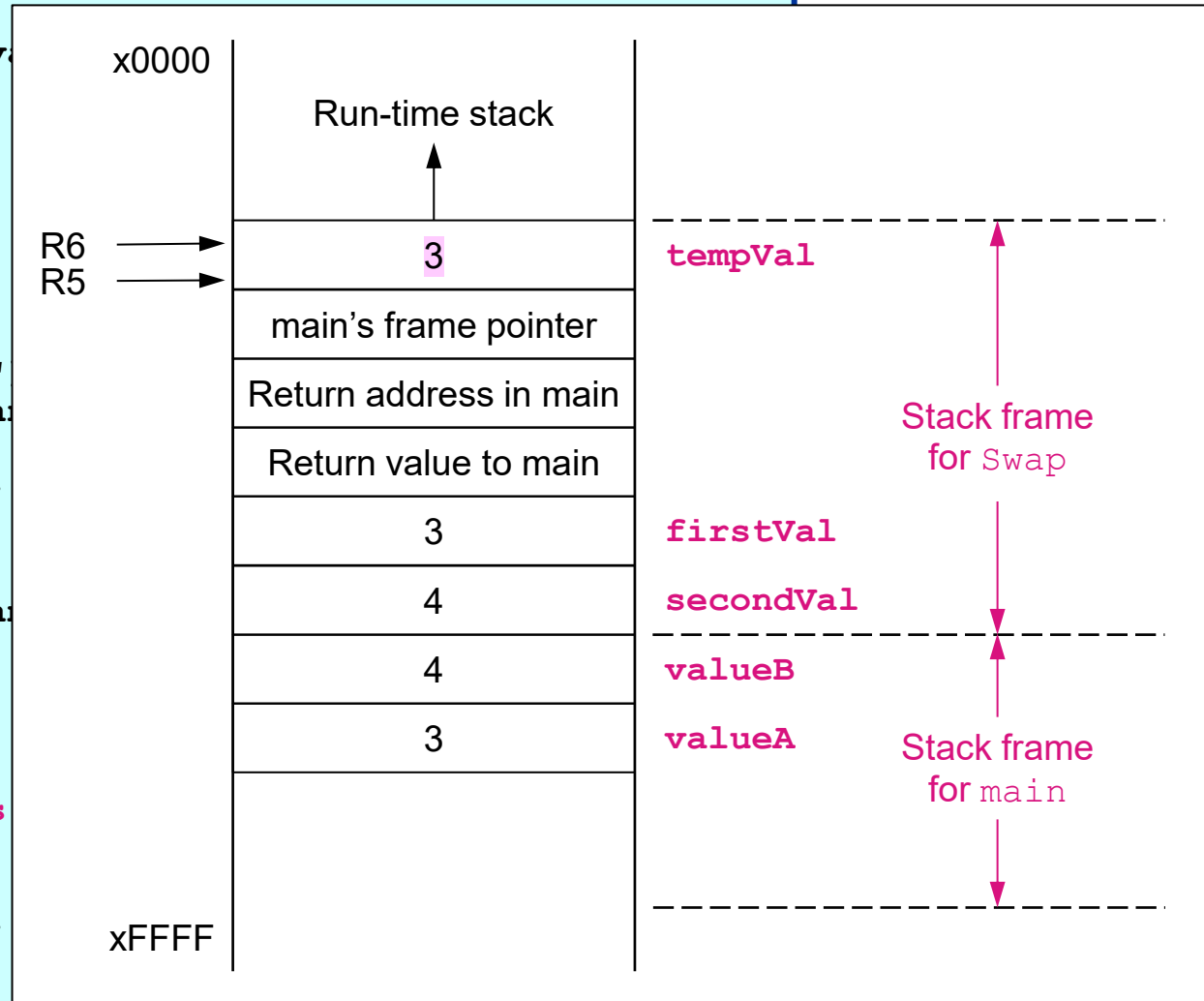
A classic example -- 'Swap'

```
1  #include <stdio.h>
2
3  // Swap has no return value
4  void Swap(int firstVal,
5            int secondVal)
6  {
7      int tempVal;
8      tempVal = firstVal;
9      firstVal = secondVal;
10     secondVal = tempVal;
11 }
12
13 int main(void)
14 {
15     int valueA = 3;
16     int valueB = 4;
17
18     printf("Before Swap\n");
19     printf("valueA = %d and valueB = %d\n", valueA, valueB);
20
21     Swap(valueA, valueB);
22
23     printf("After Swap\n");
24     printf("valueA = %d and valueB = %d\n", valueA, valueB);
25 }
26
27 void Swap(int firstVal,
28           int secondVal)
29 {
30     int tempVal; // Holds the value of firstVal
31     tempVal = firstVal;
32     firstVal = secondVal;
33     secondVal = tempVal;
34 }
```



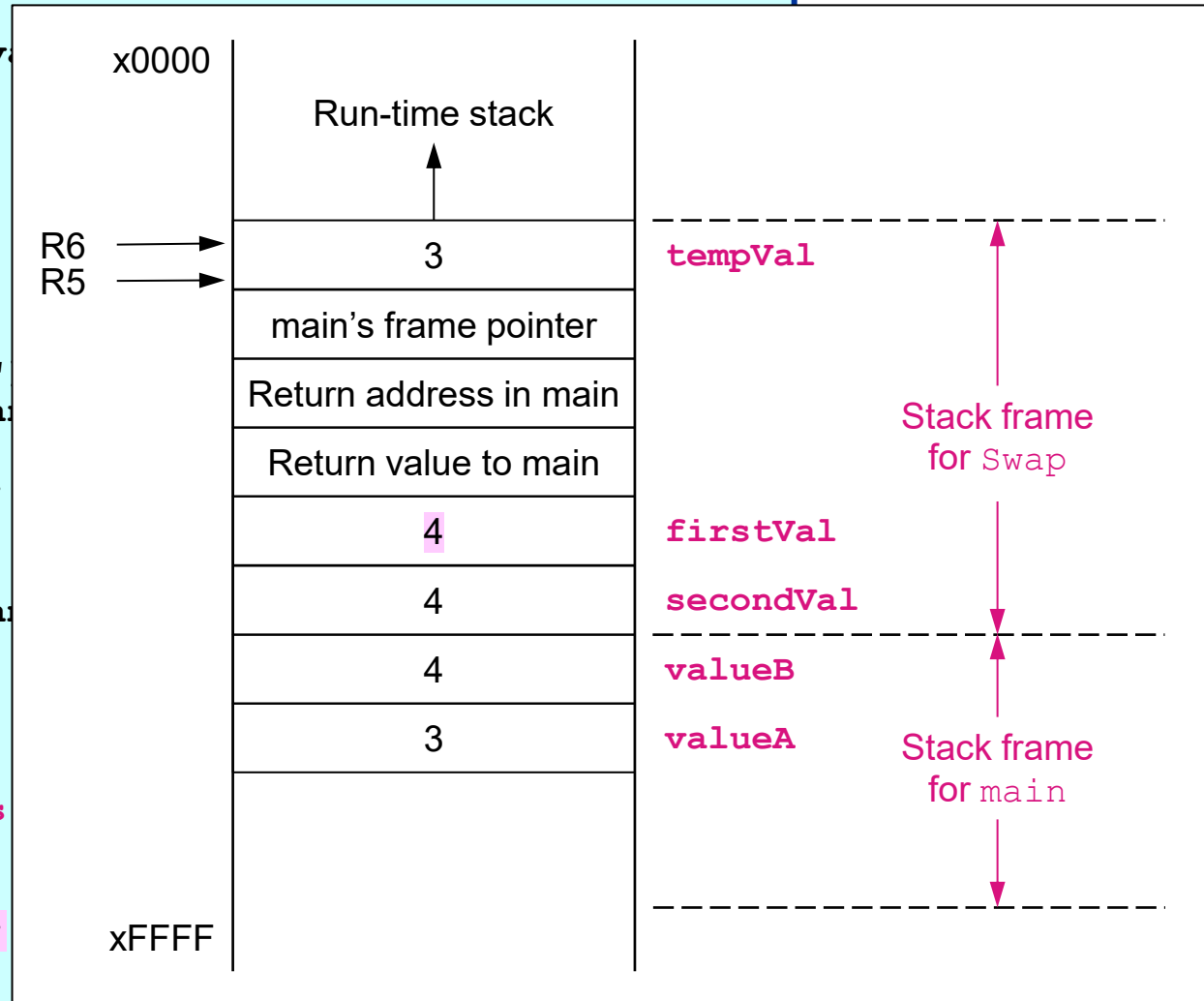
A classic example -- 'Swap'

```
1  #include <stdio.h>
2
3  // Swap has no return value
4  void Swap(int firstVal,
5           int secondVal)
6  {
7      int tempVal;
8      tempVal = firstVal;
9      firstVal = secondVal;
10     secondVal = tempVal;
11 }
12
13 int main(void)
14 {
15     int valueA = 3;
16     int valueB = 4;
17
18     printf("Before Swap\n");
19     printf("valueA = %d and valueB = %d\n", valueA, valueB);
20
21     Swap(valueA, valueB);
22
23     printf("After Swap\n");
24     printf("valueA = %d and valueB = %d\n", valueA, valueB);
25 }
26
27 void Swap(int firstVal,
28          int secondVal)
29 {
30     int tempVal; // Holds the value of firstVal
31     tempVal = firstVal;
32     firstVal = secondVal;
33     secondVal = tempVal;
34 }
```



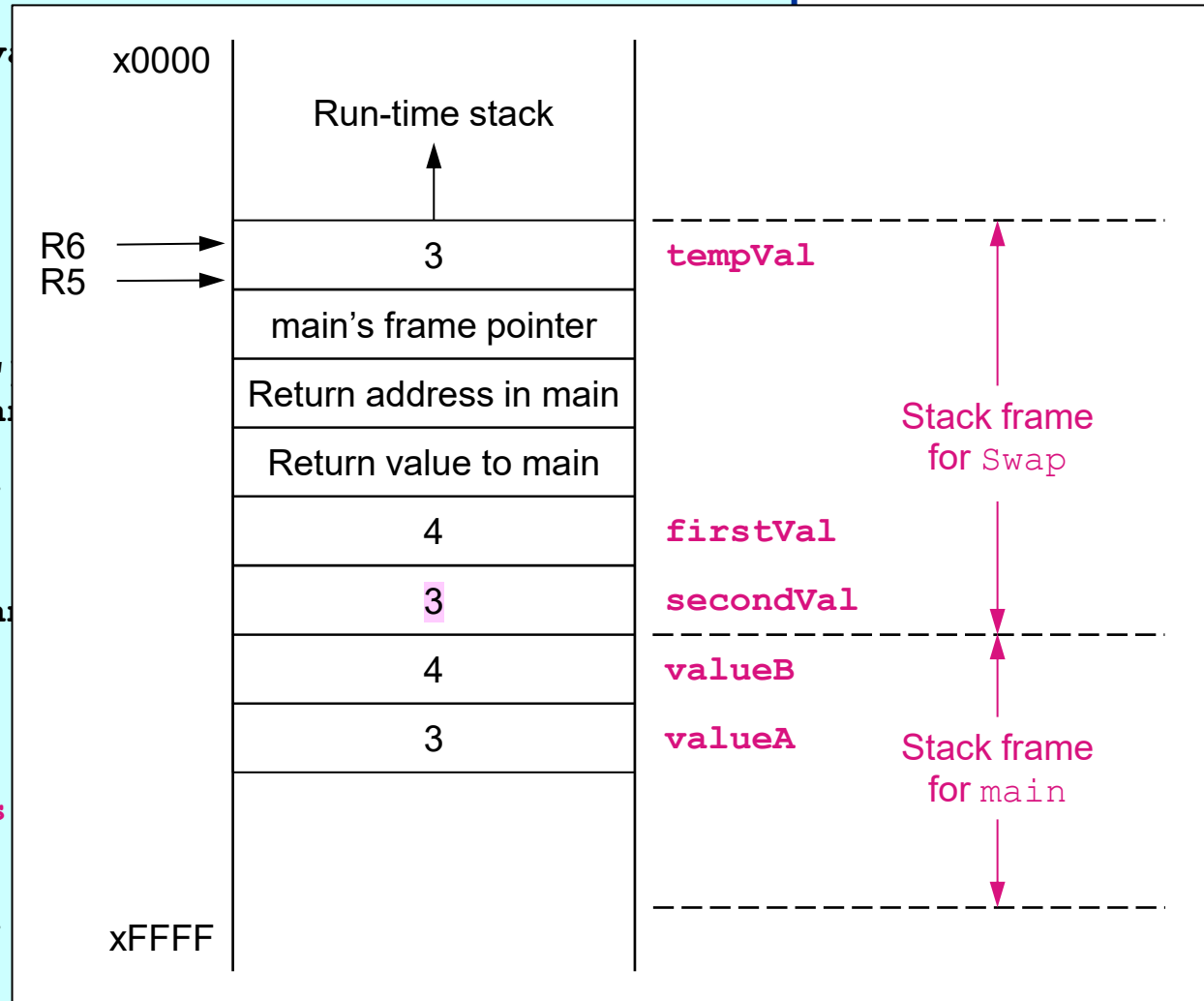
A classic example -- 'Swap'

```
1  #include <stdio.h>
2
3  // Swap has no return value
4  void Swap(int firstVal,
5            int secondVal)
6  {
7      int tempVal;
8      tempVal = firstVal;
9      firstVal = secondVal;
10     secondVal = tempVal;
11 }
12
13 int main(void)
14 {
15     int valueA = 3;
16     int valueB = 4;
17
18     printf("Before Swap\n");
19     printf("valueA = %d and valueB = %d\n", valueA, valueB);
20
21     Swap(valueA, valueB);
22
23     printf("After Swap\n");
24     printf("valueA = %d and valueB = %d\n", valueA, valueB);
25 }
26
27 void Swap(int firstVal,
28           int secondVal)
29 {
30     int tempVal; // Holds the value of firstVal
31     tempVal = firstVal;
32     firstVal = secondVal;
33     secondVal = tempVal;
34 }
```



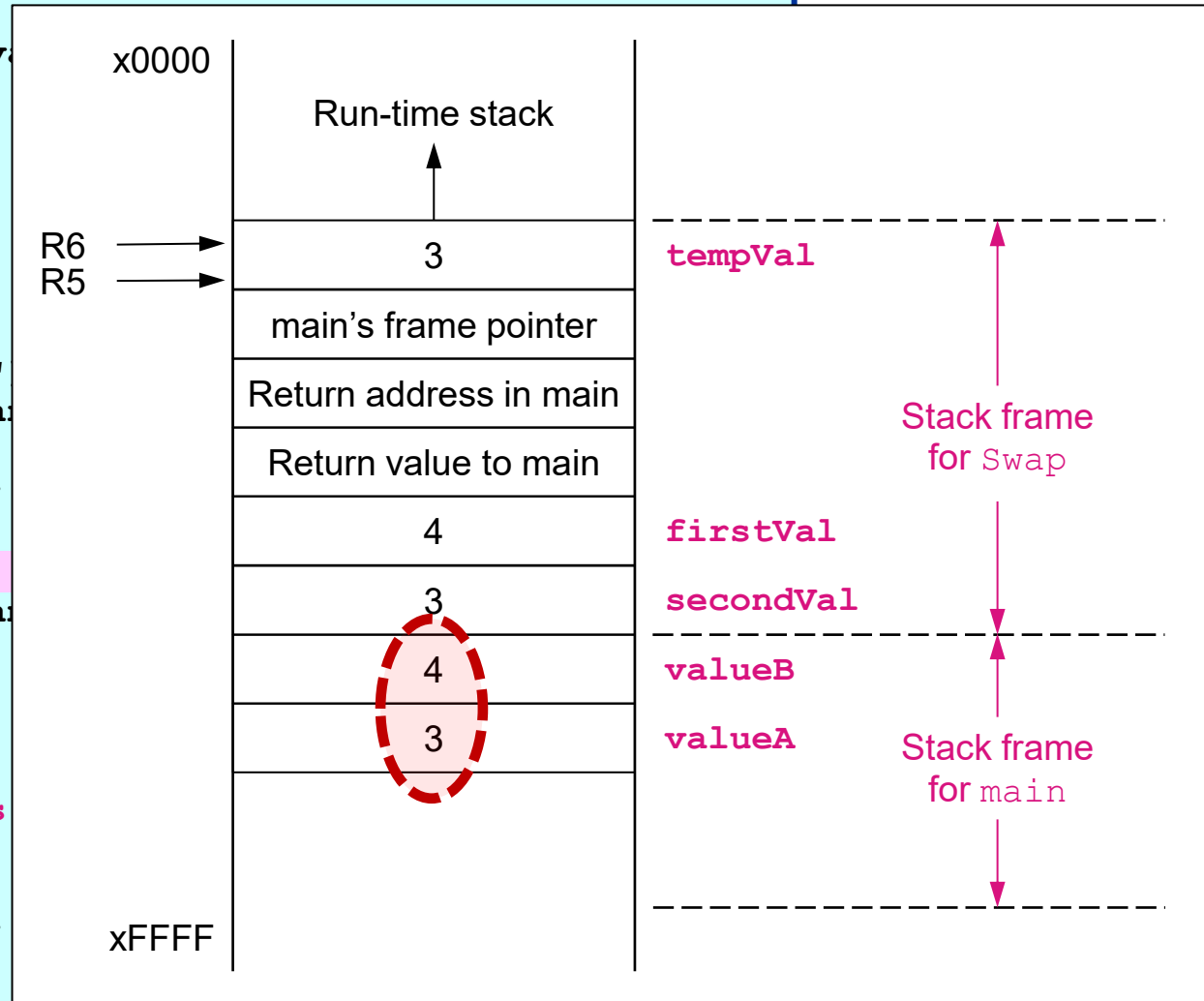
A classic example -- 'Swap'

```
1  #include <stdio.h>
2
3  // Swap has no return value
4  void Swap(int firstVal,
5            int secondVal)
6  {
7      int tempVal;
8      tempVal = firstVal;
9      firstVal = secondVal;
10     secondVal = tempVal;
11 }
12
13 int main(void)
14 {
15     int valueA = 3;
16     int valueB = 4;
17
18     printf("Before Swap\n");
19     printf("valueA = %d and valueB = %d\n", valueA, valueB);
20
21     Swap(valueA, valueB);
22
23     printf("After Swap\n");
24     printf("valueA = %d and valueB = %d\n", valueA, valueB);
25 }
26
27 void Swap(int firstVal,
28           int secondVal)
29 {
30     int tempVal; // Holds the value of firstVal
31
32     tempVal = firstVal;
33     firstVal = secondVal;
34     secondVal = tempVal;
35 }
```



A classic example -- 'Swap'

```
1  #include <stdio.h>
2
3  // Swap has no return value
4  void Swap(int firstVal,
5           int secondVal)
6  {
7      int tempVal;
8      tempVal = firstVal;
9      firstVal = secondVal;
10     secondVal = tempVal;
11 }
12
13 int main(void)
14 {
15     int valueA = 3;
16     int valueB = 4;
17
18     printf("Before Swap\n");
19     printf("valueA = %d and valueB = %d\n", valueA, valueB);
20
21     Swap(valueA, valueB);
22
23     printf("After Swap\n");
24     printf("valueA = %d and valueB = %d\n", valueA, valueB);
25 }
26
27 void Swap(int firstVal,
28          int secondVal)
29 {
30     int tempVal; // Holds the value of firstVal
31     tempVal = firstVal;
32     firstVal = secondVal;
33     secondVal = tempVal;
34 }
```



Pointers

■ Declaring Pointer Variables

- *point* to the variable whose address it contains

- Example:

- `int *ptr;`
 - `char *cp;`
 - `double *dp;`

■ C has two pointer-related operators

- Address operator: `&`
- Indirection/dereference operator: `*`

Pointer Operators (&)

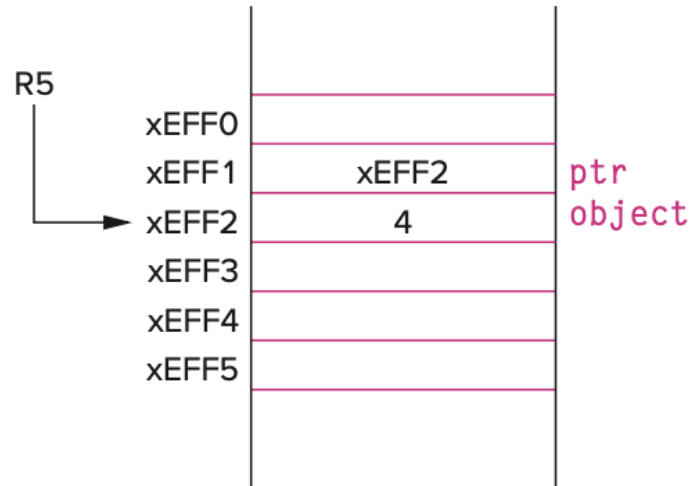
■ The address operator: &

- Generates the memory address of its operand

- Example:

```
int object;  
int *ptr;  
  
object = 4;  
ptr = &object;
```

```
AND R0,R0,#0    ; Clear R0  
ADD R0,R0,#4    ; R0 = 4  
STR R0,R5,#0    ; object = 4;  
ADD R0,R5,#0    ; Generate memory  
                ; address of object  
STR R0,R5,#-1   ; ptr = &object;
```



Pointer Operators (*)

■ The indirection/dereference operator: *

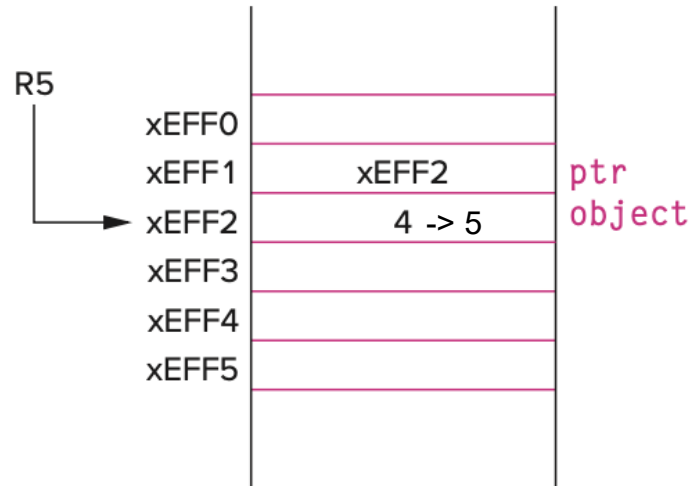
- Indirectly manipulate the value of a memory object

- Example:

```
int object;  
int *ptr;  
  
object = 4;  
ptr = &object;  
*ptr = *ptr + 1;
```

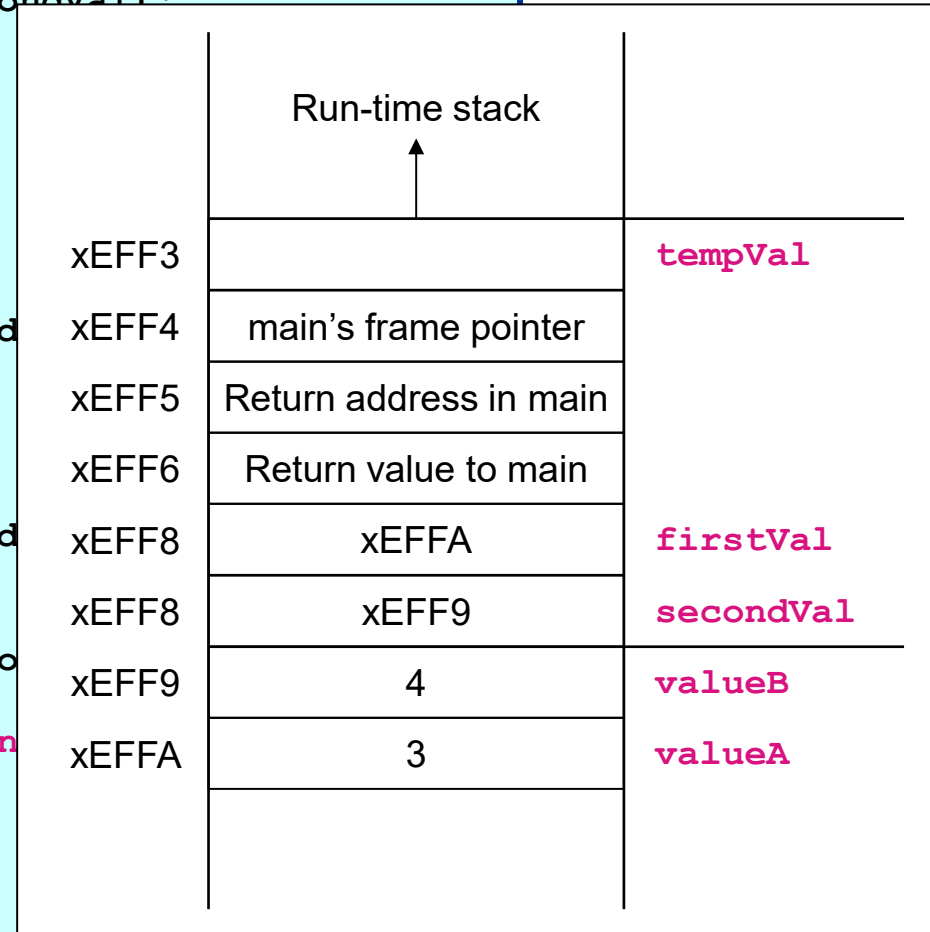
=> object = object + 1;

```
LDR R0,R5,#-1 ; R0 contains the  
               ; value of ptr  
LDR R1,R0,#0  ; R1 <- *ptr  
ADD R1,R1,#1  ; *ptr + 1  
STR R1,R0,#0  ; *ptr = *ptr + 1;
```



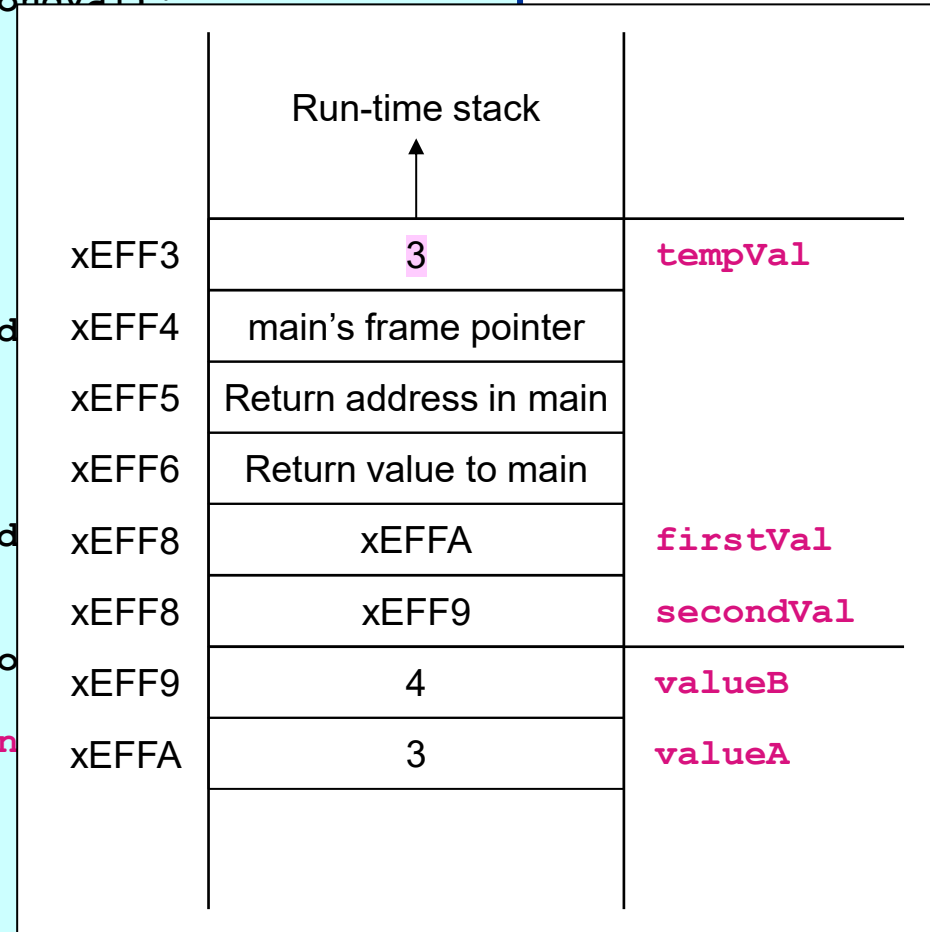
Passing a Reference Using Pointers

```
1  #include <stdio.h>
2
3  void NewSwap(int *firstVal, int *secondVal) {
4
5  int main(void)
6  {
7      int valueA = 3;
8      int valueB = 4;
9
10     printf("Before Swap ");
11     printf("valueA = %d and valueB = %d\n", valueA, valueB);
12
13     NewSwap(&valueA, &valueB);
14
15     printf("After Swap ");
16     printf("valueA = %d and valueB = %d\n", valueA, valueB);
17 }
18
19 void NewSwap(int *firstVal, int *secondVal) {
20 {
21     int tempVal; // Holds firstVal when swapped
22
23     tempVal = *firstVal;
24     *firstVal = *secondVal;
25     *secondVal = tempVal;
26 }
```



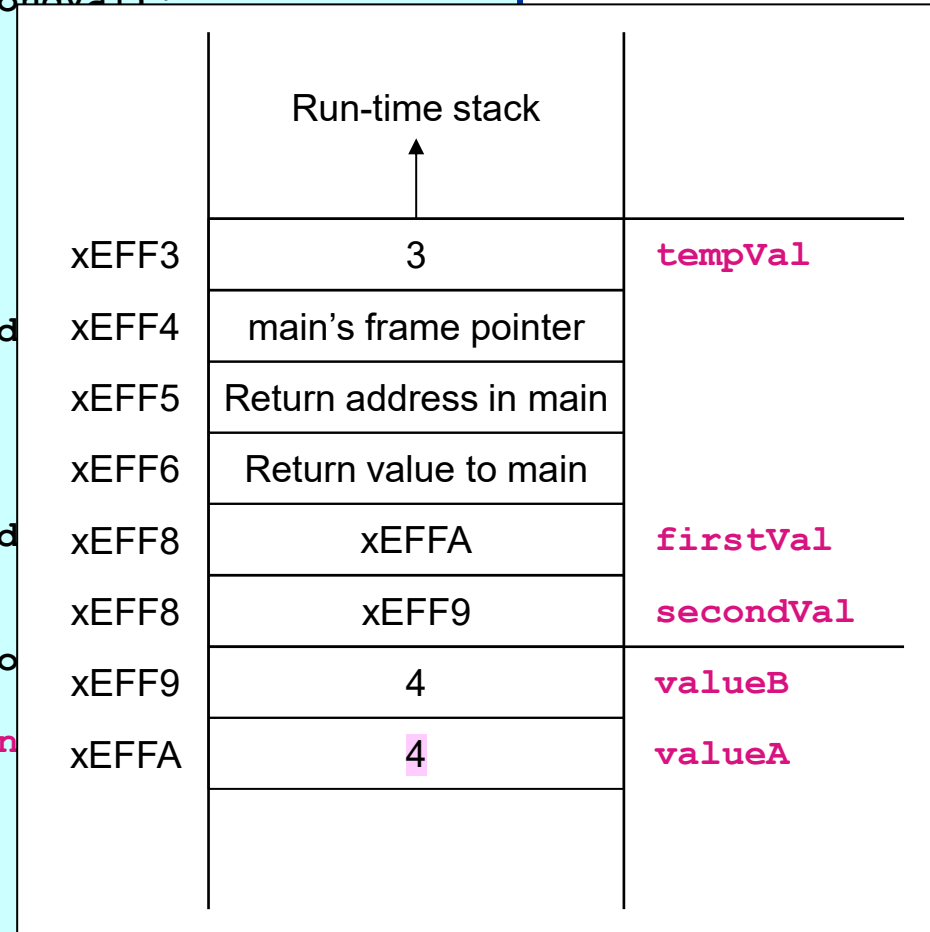
Passing a Reference Using Pointers

```
1  #include <stdio.h>
2
3  void NewSwap(int *firstVal, int *secondVal) {
4
5  int main(void)
6  {
7      int valueA = 3;
8      int valueB = 4;
9
10     printf("Before Swap ");
11     printf("valueA = %d and valueB = %d\n", valueA, valueB);
12
13     NewSwap(&valueA, &valueB);
14
15     printf("After Swap ");
16     printf("valueA = %d and valueB = %d\n", valueA, valueB);
17 }
18
19 void NewSwap(int *firstVal, int *secondVal) {
20 {
21     int tempVal; // Holds firstVal when swapped
22
23     tempVal = *firstVal;
24     *firstVal = *secondVal;
25     *secondVal = tempVal;
26 }
```



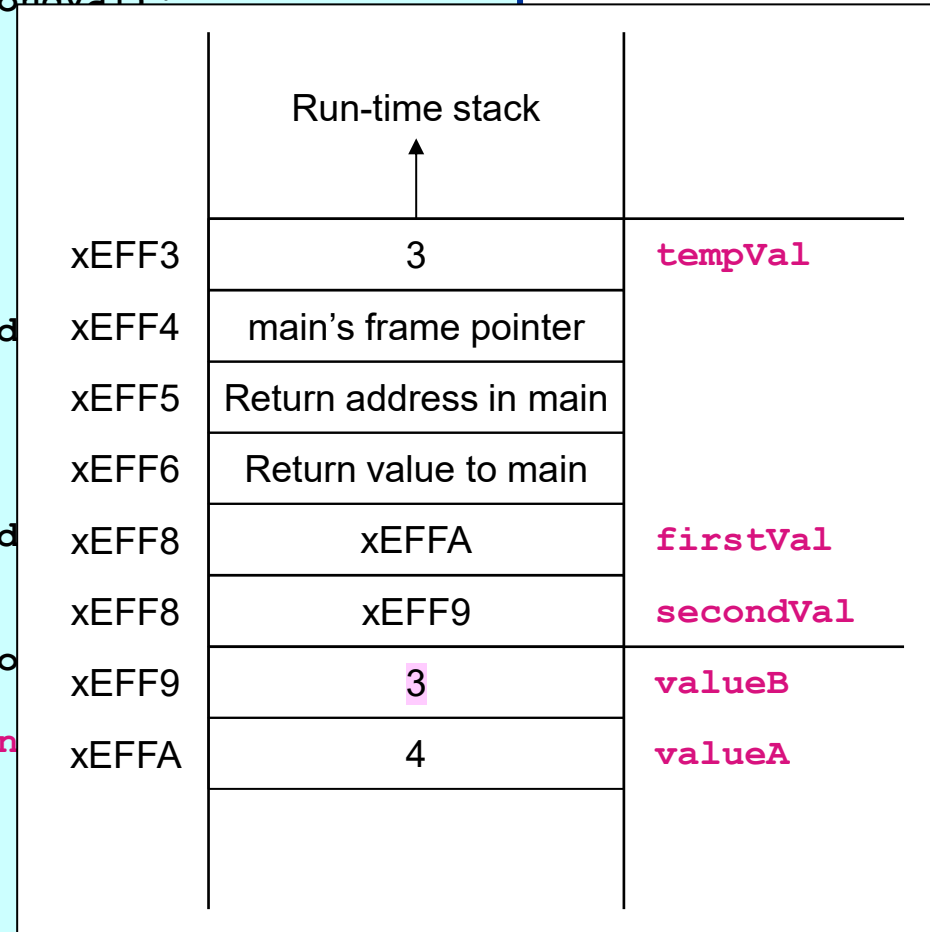
Passing a Reference Using Pointers

```
1  #include <stdio.h>
2
3  void NewSwap(int *firstVal, int *secondVal) {
4
5  int main(void)
6  {
7      int valueA = 3;
8      int valueB = 4;
9
10     printf("Before Swap ");
11     printf("valueA = %d and valueB = %d\n", valueA, valueB);
12
13     NewSwap(&valueA, &valueB);
14
15     printf("After Swap ");
16     printf("valueA = %d and valueB = %d\n", valueA, valueB);
17 }
18
19 void NewSwap(int *firstVal, int *secondVal) {
20 {
21     int tempVal; // Holds firstVal when swapped
22
23     tempVal = *firstVal;
24     *firstVal = *secondVal;
25     *secondVal = tempVal;
26 }
```



Passing a Reference Using Pointers

```
1  #include <stdio.h>
2
3  void NewSwap(int *firstVal, int *secondVal) {
4
5  int main(void)
6  {
7      int valueA = 3;
8      int valueB = 4;
9
10     printf("Before Swap ");
11     printf("valueA = %d and valueB = %d\n", valueA, valueB);
12
13     NewSwap(&valueA, &valueB);
14
15     printf("After Swap ");
16     printf("valueA = %d and valueB = %d\n", valueA, valueB);
17 }
18
19 void NewSwap(int *firstVal, int *secondVal) {
20 {
21     int tempVal; // Holds firstVal when swapped
22
23     tempVal = *firstVal;
24     *firstVal = *secondVal;
25     *secondVal = tempVal;
26 }
```





1 Review

2 Pointers

3 **Arrays**

4 Summary

Arrays

■ Declaring Arrays

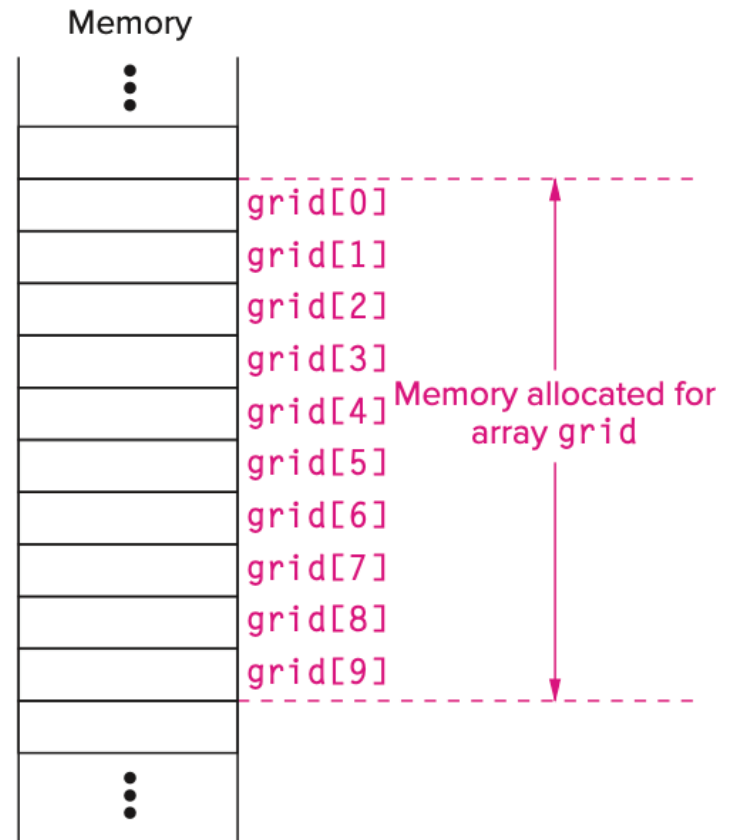
- Contiguous sequence of values

- Example:

```
-int grid[10];
```

- *grid* allocated in memory

```
-grid[0]...grid[9]
```



Using Arrays – Example 1

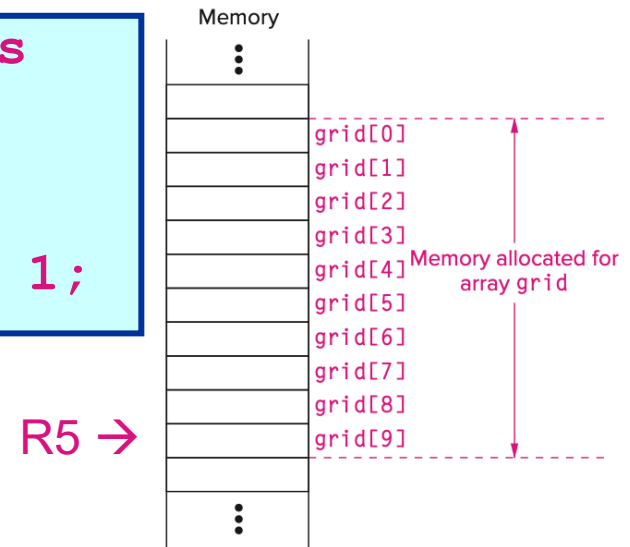
■ The power of arrays comes from

- compute an array index using a C integral expression

■ Example 1:

- `grid[6] = grid[3] + 1;`

```
ADD R0,R5,#-9 ; Put the base address  
                ; of grid into R0  
LDR R1,R0,#3   ; R1 <-- grid[3]  
ADD R1,R1,#1   ; R1 <-- grid[3] + 1  
STR R1,R0,#6   ; grid[6] = grid[3] + 1;
```

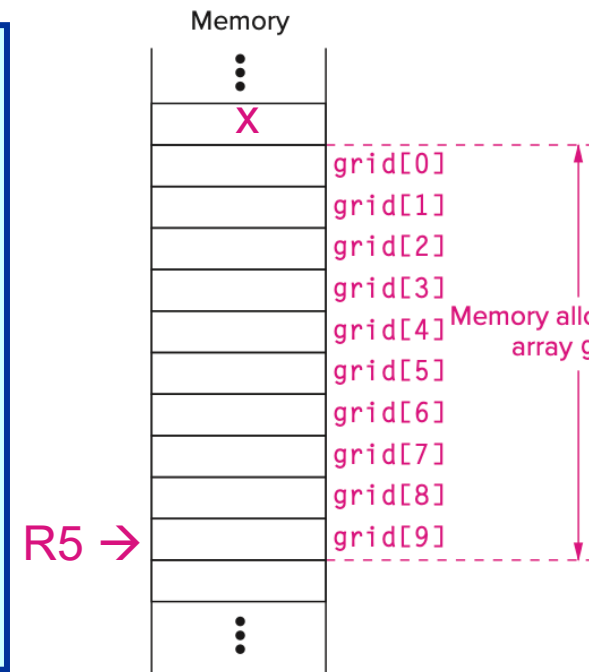


Using Arrays – Example 2

■ Example 2:

- $\text{grid}[\text{x}+1] = \text{grid}[\text{x}] + 2;$

```
LDR R0,R5,#-10 ; Load the value of x
ADD R1,R5,#-9  ; Put the base address
                ; of grid into R1
ADD R1,R0,R1   ; Calc. addr. of grid[x]
LDR R2,R1,#0   ; R2 <-- grid[x]
ADD R2,R2,#2   ; R2 <-- grid[x] + 2
LDR R0,R5,#-10 ; Load the value of x
ADD R0,R0,#1   ; R0 <-- x + 1
ADD R1,R5,#-9  ; Put the base address
                ; of grid into R1
ADD R1,R0,R1   ; Calc. addr. of grid[x+1]
STR R2,R1,#0   ; grid[x+1] = grid[x] + 2;
```



Arrays as parameters

■ Passing arrays between functions

- option 1: pass the entire array of values
- option 2: pass a reference to the array
- If the array contains a large number of elements
 - copying each element from one stack frame onto another could be very costly in execution time
- C naturally passes arrays by reference

Arrays as parameters

■ Passing arrays between functions

● Example:

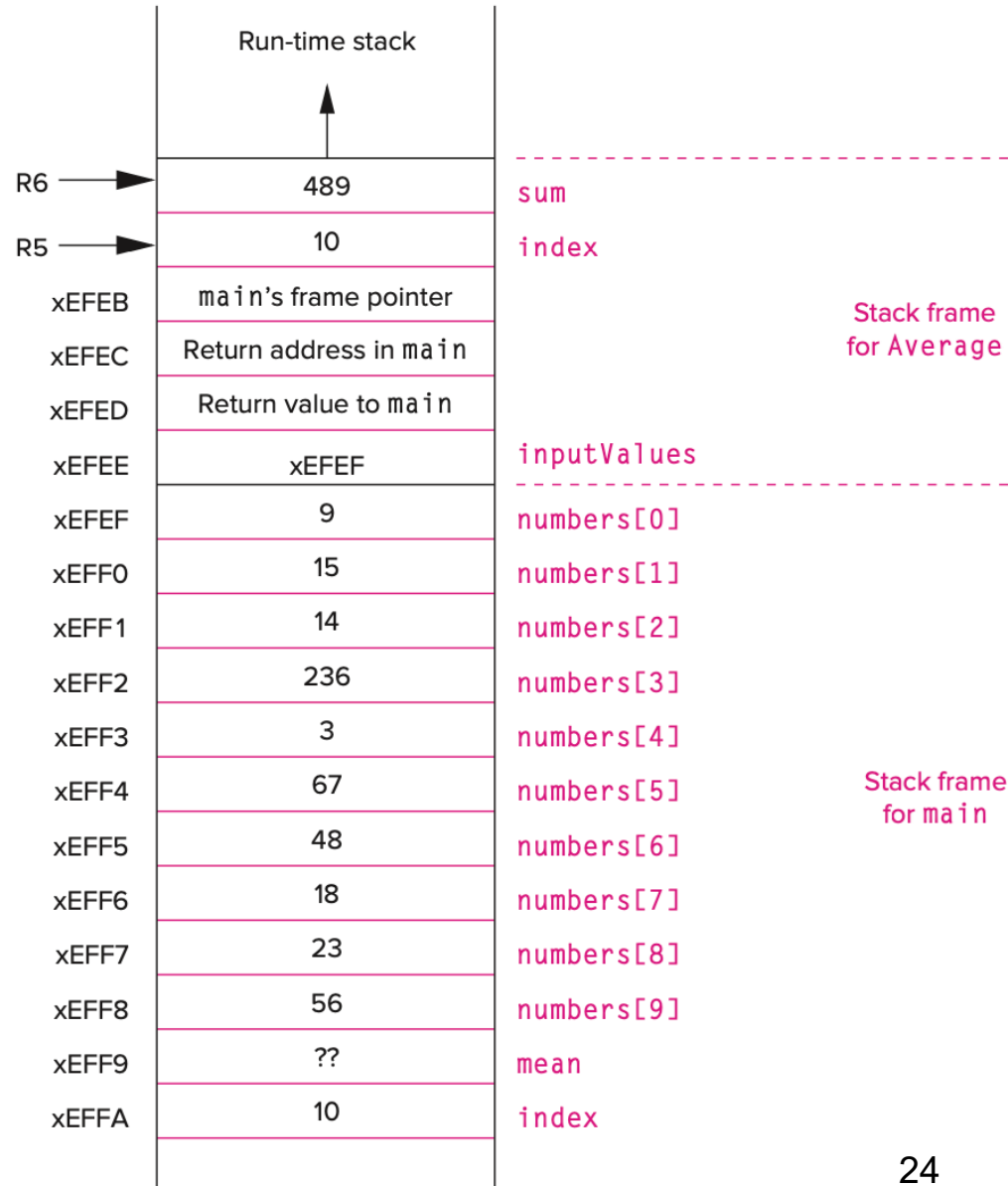
```
1  #include <stdio.h>
2  #define MAX_NUMS 10
3  int Average(int input_values[]);
4  int main(void)
5  {
6      int mean;                // Average of numbers
7      int numbers[MAX_NUMS];  // Original input numbers
8
9      // Get input
10     // ...
11
12
16     mean = Average(numbers);
17     printf("The average of these numbers is %d\n", mean);
18 }
19
20 int Average(int inputValues[])
21 {
22     int sum = 0;
23     for (int index = 0; index < MAX_NUMS; index++) {
24         sum = sum + inputValues[index];
25     }
26     return (sum / MAX_NUMS);
27 }
```

Arrays as parameters

```

1  #include <stdio.h>
2  #define MAX_NUMS 10
3  int Average(int input_values[]);
4  int main(void)
5  {
6      int mean;           // Average
7      int numbers[MAX_NUMS]; // Original array
8
9      // Get input
10     // ...
11
12     mean = Average(numbers);
13     printf("The average of these numbers is %d\n", mean);
14 }
15
16 int Average(int inputValues[])
17 {
18     int sum = 0;
19     for (int index = 0; index < MAX_NUMS; index++)
20         sum = sum + inputValues[index];
21 }
22 return (sum / MAX_NUMS);
23 }

```



Strings in C

■ Strings are sequences of characters that represent text

● Example:

– `char word[10];`

● Array *word* can contain the strings

– “elephant” or “giraffe”, but not the entirety of “hippopotamus”

● The null character – ‘\0’ (ASCII value: 0)

– Such strings are also called *null-terminated strings*.

```
char word[10];  
  
word[0] = 'g';  
word[1] = 'i';  
word[2] = 'r';  
word[3] = 'a';  
word[4] = 'f';  
word[5] = 'f';  
word[6] = 'e';  
word[7] = '\0';  
printf("%s", word);
```

```
char word[10] = "Hello";  
  
printf("%s", word);
```

Arrays vs. Pointers in C

■ Example:

```
char word[10];  
char *cptr;  
cptr = word;
```

- Access the fourth character within the string
– *word[3]* or **(cptr + 3)* or *cptr[3]*
- *cptr* is a variable and can be reassigned
- *word* is an array and cannot be reassigned

	Using a Pointer	Using Name of Array	Using Array Notion
Address of array	cptr	word	&word[0]
0th element	*cptr	*word	word[0]
Address of element n	(cptr + n)	(word + n)	&word[n]
Element n	*(cptr + n)	*(word + n)	word[n]

Variable-Length Arrays

- Arrays whose sizes are variable integer expressions, rather than constants (C99 standard)

- Example:

```
int Calculate(int len)
{
    int data[len];
    // ...
}
```

- The size of the array data can only be determined during execution of the code, and it is not known at compile time.
- Array *data* is still allocated on the run-time stack
 - requires additional instructions during allocation and access
- More general solution:
 - malloc/free, new[] / delete[]



1 Review

2 Pointers

3 Arrays

4 Summary

Summary

■ Two programming constructs: Pointers and arrays

- Enable us to access memory *indirectly*

■ Pointers

- Variables that contain address of other memory objects
- Indirectly access and manipulate these other objects
 - Pass parameters by reference
 - Construct complex data structure

■ Arrays

- A collection of elements that the same type arranged *sequentially* in memory
- Access a particular element within an array by providing an index to the element that is its offset from the beginning of the array
- A significant structure for organizing data
 - Character string, vector, tensor, stack, queue, ...

Exercise

16.11 For this question, refer to the following C program:

```
int FindLen(char *);
int main(void)
{
    char str[10];

    printf("Enter a string : ");
    scanf("%s", str);
    printf("%s has %d characters\n", str, FindLen(str));
}

int FindLen(char * s)
{
    int len=0;
    while (*s != '\0') {
        len++;
        s++;
    }
    return len;
}
```

- What is the size of the stack frame for the functions `main` and `FindLen`?
- Show the contents of the stack just before `FindLen` returns if the input string is `apple`.
- What would the stack frame look like if the program were run and the user typed a string of length greater than ten characters? What would happen to the program?